

IBM ECM System Monitor Field Guides

Deploying ESM as Container – Docker, Kubernetes or Openshift

August 4th, 2026
CENIT AG
Michael Wohland

Content

Introduction	3
Prerequisites	4
ECM System Monitor	4
OpenShift (specifics)	4
Client Tools	4
First Step - Download ESM Container Images	5
Second Step - Import Container Images in Container Registry	6
Example using docker – Use equivalent tools for your cluster type:	6
Third Step - Add Helm Chart Repository (Kubernetes and Openshift only)	7
Fourth Step – Deploying Containers / Installation	8
Example deployment using Helm (Kubernetes and Openshift only)	8
Example deployment using Docker (Docker only)	14
Fifth Step - Validation	15
Check Resource States	15
Check Logs	15
Show Pods	16
Access ESM Server UI Using Port Forwarding	17
Uninstallation	18
Contact Information	19

Introduction

This guide will provide step-by-step deployment information. All information will use basic tools. In addition, examples will be provided.

You may need to adopt the commands to fit your container environment. As a requirement you need to know how to work within your container environment e.g.

- Authenticating to the API of your private cluster
- Providing container images through a private registry
- Configuring DNS and load balancers for accessing the ESM UI from outside the cluster.

The deployment is performed using a Helm Chart. An ESM instance installed with Helm (a “Helm Release”) can consist of:

- 0 or 1 ESM Servers
- 0 or 1 ESM Agents

For most scenarios, a single release consisting of 1 ESM Server and 1 ESM Agent is sufficient for typical environments to be monitored within a cluster.

Prerequisites

ECM System Monitor

This guide is for customers running IBM ECM System Monitor (ESM) V5.8.0, released in July 2026. The container images and the Helm Chart are continuously improved with each release of ESM. If required, this guide will be updated to reflect these changes.

OpenShift (specifics)

ECM System Monitor is supported on OpenShift (OCP) 4.x only. OpenShift 3.x versions are not supported.

Client Tools

- **helm** needs to be available (v3+). Helm 2.x is not supported. (OpenShift and Kubernetes deployments only)
- Command line tool compatible with your cluster's version is recommended, e.g. **oc** or **az**.
- To push the required container images to a Container Registry (such as the "IBM Container Registry"), you may use your preferred container runtime (such as **docker** or **podman**) or other tools (**buildah** or **skopeo**). In this document we will provide instructions for **docker**.

First Step - Download ESM Container Images

The ESM Containers are not part of any container registry. The container files must be downloaded and imported manually into your “private” container registry.

The containers are available via:

- IBM Password Advantage (major and minor release versions)
- IBM Fix Central (feature pack and interim fix versions)

Although IBM Fix Central only provides ESM feature pack and interim fix versions, these containers are fully functional and do not require a container of a major or minor release version. Nevertheless, for the ESM Server it may be required to use certain versions due to Database updates that have to be performed when updating the system.

For better readability and since the IF packages also use this naming convention, please rename the tar.gz files after download to `esmserver.tar.gz` and `esmagent.tar.gz`.

Once done please extract the gz archive so you receive a tar file:

- ESM Server: `esmserver.tar.gz` -> `esmserver.tar`
- ESM Agent: `esmagent.tar.gz` -> `esmagent.tar`

If the extracted tar file does not have this name – please rename again.

Second Step - Import Container Images in Container Registry

Example using docker – Use equivalent tools for your cluster type:

- 1) Setup access to the registry. This may require authentication with e.g. an IBM Cloud or Azure Cloud account.
Notice: Your account needs to be member of a Resource Group which has permissions to use the “Container Registry Service”.
- 2) Import both images separately to your local container runtime:

```
docker load --input esmsserver.tar
docker load --input esmagent.tar
```

- 3) This step is optional as you might already have a namespace in a registry. If not, create a namespace for the images in the registry. In this example we will use a private namespace called **cenit** in the **de.icr.io** registry (this registry/namespace will not be accessible to you).
- 4) Tag the images for the registry (replace registry/namespace “de.icr.io/cenit” with your registry/namespace – also use the correct version number of ESM for the tag – V = release version, W = major version, X = minor version, Y = feature pack version and Z = interims fix version e.g. 5.8.0.0-000):

```
docker tag esmsserver:V.W.X.Y-Z de.icr.io/cenit/esmsserver:V.W.X.Y-Z
docker tag esmagent:V.W.X.Y-Z de.icr.io/cenit/esmagent:V.W.X.Y-Z
```

- 5) Login to Container Registry.
- 6) Push the tagged images:

```
docker push de.icr.io/cenit/esmsserver:V.W.X.Y-Z
docker push de.icr.io/cenit/esmagent:V.W.X.Y-Z
```

- 7) List the pushed images:

```
ibmcloud cr image-list
```

Output (truncated):

Repository	Tag	Digest	Size
de.icr.io/cenit/esmagent	V.W.X.Y-Z	a7941079a4c7	367 MB
de.icr.io/cenit/esmsserver	V.W.X.Y-Z	6d1b52f06af3	281 MB

The images are available in your registry namespace now.

Third Step - Add Helm Chart Repository (Kubernetes and Openshift only)

You can get the chart by adding the Helm repository:

cenit-ag.github.io/helm-charts

Run the following commands to add the repository to your Helm environment:

```
# Add repository
helm repo add cenit https://cenit-ag.github.io/helm-charts

# Update the repository index
helm repo update
```

Available versions of the chart can be listed with a search:

```
helm search repo cenit
```

Output:

NAME	CHART	VERSION	[...]
cenit/sm		2.0.5	[...]

If the client you will be sending your commands from does not have an internet connection, you can pull the chart:

```
helm pull cenit/sm --version 2.0.5
```

This will download the chart as a .tgz file which can be used for an offline installation.

Notice: The --version flag is optional. If no version is stated, always the latest version will be downloaded.

The Helm chart repository also included extensive chart documentation. See following as an example for a specific chart version (v2.0.5):

<https://github.com/cenit-ag/helm-charts/tree/main/sm/docs>

Notice: Instead of helm command for downloading the chart, you can also download the chart from this git repository and extract it to your system:

<https://github.com/cenit-ag/helm-charts/tree/main/sm>

Fourth Step – Deploying Containers / Installation

Example deployment using Helm (Kubernetes and Openshift only)

The deployment can be done using a simple helm command:

```
helm install mysm cenit/sm -f values.yaml -n esm
```

- **mysm** is the release name and will also be used for the agent name for the connection to the ESM server.
- **cenit/sm** points to the chart in the repository we added in Step 3 in this guide. This can also be replaced with the path to a **.tgz** archive containing the chart or a directory with the chart sources from within the **.tgz** archive.
- **values.yaml** is the values file that contains adjustment values for the Helm chart deployment – examples follow below.
- **-n esm** namespace in which the deployment should be done – make sure to deploy the agent into the namespace where it will monitor your Business Automation systems.
- Optionally the flag **--dry-run** can be added to simulate the Helm templating procedure on the client side without committing any changes to the targeted cluster. This is helpful in case you want to validate if your values YAML file (**values.yaml** in the example) is syntactically correct.

Helm offers the possibility to install a server and agent container at the same time. This can be specified in the values.yaml.

The command **helm list** will show all deployed releases.

Command **helm status mysm** will give the current status of the chart including the post-installation notes, which include some helpful commands for further troubleshooting.

Notice: After a successful Helm installation, the “NOTES” passage in the output features several **kubect1** commands to validate the installation. You can replace **kubect1** with **oc** to run these commands without having **kubect1** available on your command line – also see next chapter.

Example values.yaml files for deployment with Helm

Notice: The values.yaml for Kubernetes and OpenShift deployment is the same. Only for OpenShift it requires platform information which is shown in the first example.

Minimum values.yaml Server and Agent deployment:

For deploying ESM a minimum set of parameters (image repository and image tag) in the values.yaml is required – again, please make sure to replace repository and tag with the correct information.

Content of values.yaml:

```
platform: ocp

global:
  imagePullSecrets:      # Optional; default is 'pull-secret-to-be-used'.
    - name: dummy-secret # Replace it by your own secret!
  imagePullPolicy: Always # Optional; default is 'IfNotPresent'.
  imageTag: 5.8.0.0-000   # Required.

server:
  image:
    repository: company-registry.example.com/sm/smserver

agent:
  image:
    repository: company-registry.example.com/sm/smagent
```

Minimum values.yaml Agent deployment only:

If you do not want to deploy the ESM Server you have to disable it via the parameters:

Content of values.yaml:

```
global:
  imageTag: 5.8.0.0-000

server:
  enabled:
    false

agent:
  image:
    repository: company-registry.example.com/sm/smagent
```

values.yaml Agent deployment connect to specific ESM server host:

If the agent should connect to an ESM Server that is not deployed within the same release name, you need to add the `serverHostname` parameter to the agent section. Make sure the required firewall ports are open. Specify the hostname either full qualified or IP-address:

Content of values.yaml:

```
server:
  [...]
agent:
  [...]
  serverHostname: myESMServerName.myDomain
```

values.yaml Agent adding secure (SSL) CPE certs to deployment:

For monitoring CPE information via SSL URL it is required to import certificates in the cacerts truststore of the agent JRE. As we do not want to do that within the container we want to mount the cacerts file as a secret during deployment and therefore we need to specify some additional parameters:

You may already have a company wide cacerts secret, if not

- Get a copy of the cacerts and add all required certificates (root/inter/host).
- Create a secret e.g. with `kubectl` or `oc` command (remember to redo that if certificates change):

```
kubectl/oc create secret generic sm-cacerts --from-file=cacerts
```

Content of values.yaml:

```
server:
  [...]
  # ----- #
  # 'caCerts' and 'certificate' are mutually exclusive! #
  # ----- #
  caCerts:
    secretName: company-cacerts

agent:
  [...]
  # ----- #
  # 'caCerts' and 'certificate' are mutually exclusive! #
  # ----- #
  caCerts:
    secretName: company-cacerts
```

values.yaml Server with external DB:

To run an ESM server container with an external database server like DB2 or MSSQL you need to create the secret(s) for the Database Credentials first:

```
Kubect1/oc create secret generic creds-conf-db --from-literal=user='YourUsr' --from-literal=password='YourPW'

Kubect1/oc create secret generic creds-mon-db --from-literal=user='YourUsr' --from-literal=password='YourPW'
```

Content of values.yaml for DB2:

```
server:
  [...]
  externalConfigDatabase:
    enabled: true
    jdbcUrl: "jdbc:db2://mydb2:50000/smconfdb"
    jdbcDriverClass: "com.ibm.db2.jcc.DB2Driver"
    jdbcSecret: creds-conf-db
  externalMonitoringDatabase:
    enabled: true
    jdbcUrl: "jdbc:db2://mydb2:50000/smmondb"
    jdbcDriverClass: "com.ibm.db2.jcc.DB2Driver"
    jdbcSecret: creds-mon-db
agent:
  [...]
```

Content of values.yaml for MSSQL:

```
server:
  [...]
  externalConfigDatabase:
    enabled: true
    jdbcUrl: "jdbc:sqlserver://mysqlserver:1433;databaseName=smconfdb"
    jdbcDriverClass: "com.microsoft.sqlserver.jdbc.SQLServerDriver"
    jdbcSecret: creds-conf-db

  externalMonitoringDatabase:
    enabled: true
    jdbcUrl: "jdbc:sqlserver://mysqlserver:1433;databaseName=smmondb"
    jdbcDriverClass: "com.microsoft.sqlserver.jdbc.SQLServerDriver"
    jdbcSecret: creds-mon-db
agent:
  [...]
```

Preparations for Logfile Monitoring with Helm

The following shows the steps that are necessary to monitor CPE and ICN logfiles by an ESM Agent pod.

The basic principles apply to all logfiles that should be monitored directly by an ESM Agent pod. The logfiles to be monitored must be accessible by a Persistent Volume Claim (PVC) and the agent's deployment must then reference that PVC to be able to mount it in its own filesystem hierarchy.

Newer versions of ICN and CPE deployments are providing PVCs and Persistent Volumes (PV) for the log folders of CPE and ICN pods. All ICN and CPE pods in the deployment share the PVs for the logfiles.

The following table shows examples of PVCs and PVs for CPE and ICN deployments.

Product	Type	Name	Mount Path in Agent Container
CPE	PVC	cpe-fnlogstore	/opt/sm/external-volumes/<claim-name>
ICN	PVC	icn-fnlogstore	/opt/sm/external-volumes/<claim-name>
...	...	...	...

The agent deployment must include volume definitions in the **volumes** section so the PVCs can be referenced in the **volumeMounts** section for mounting. Here is an example of the **volume claims** section in the agent's deployment YAML. The entries for these volume claims and volumes are highlighted in bold for better readability only.

```
agent:
  [...]
  volumeClaims:
    - name: my-cpe-logs
      claimName: cpe-fnlogstore
    - name: my-navigator-logs
      claimName: icn-fnlogstore
  [...]
```

For the log monitoring itself you can use the given mount paths and enhance that with logfile names. As containers use specific log names, you probably must use wild cards. The logfile mechanism only reads logs which have changed within the last 72 hours.

Complex values.yaml with typical requirements with Helm

The following example covers the most used features for a values.yaml

```
#StartOfFile-----

platform: ocp

global:
  imagePullSecrets:      # Optional; default is 'pull-secret-to-be-used'.
    - name: dummy-secret # Replace it by your own secret!
  imagePullPolicy: Always # Optional; default is 'IfNotPresent'.
  imageTag: 5.8.0.0-000   # Required.

server:
  image:
    repository: company-registry.example.com/sm/smagent # Required.
  caCerts:
    secretName: company-cacerts
  singleVolumePersistence:
    enabled: true
    existingClaim: sm-global-pvc
  persistence:
    enabled: false
    existingClaim: sm-server-data-pvc
  agentsSync:
    enabled: false
    existingClaim: sm-server-agent-sync-pvc
  externalDependenciesVolume:
    enabled: false
    existingClaim: sm-external-dependencies-pvc

agent:
  image:
    repository: company-registry.example.com/sm/smagent # Required.
  singleVolumePersistence:
    enabled: true
    existingClaim: sm-global-pvc
  volumeClaims:
    - name: my-cpe-logs
      claimName: cpe-fnlogstore
    - name: my-navigator-logs
      claimName: icn-fnlogstore
  caCerts:
    secretName: company-cacerts

#EndOfFile-----
```

Additional Information for values.yaml

Please review the latest readme for further information:
<https://github.com/cenit-ag/helm-charts/tree/main/sm/docs>

Example deployment using Docker (Docker only)

If you are running the containers in a Docker environment, the Helm install command is typically not available. Instead, you will run the container with “docker run” command. Here are some examples; the network **smnet** is an example and must be created manually beforehand.

Server:

```
docker run -d --name smserver --net=smnet -p 8080:8080 -p 8443:8443 -p 1883:1883  
esmserver:5.8.0.0-000
```

Agent:

```
docker run -d --name smagent --net=smnet -p 1180:1180 -e  
SERVER_HOSTNAME='smserver' -e CLIENT_ID='DkrAgent' esmagent:5.8.0.0-000
```

```
docker run -d --name smagent_2 --net=smnet -p 1181:1180 -e  
SERVER_HOSTNAME='smserver' -e CLIENT_ID='DkrAgent_2' esmagent:5.8.0.0-000
```

Notice: External ports are mapped to internal ports. Running several agents at the same time is possible, only specify different name, external port and CLIENT_ID. The CLIENT_ID will be used as Agent unique name in the console.

Fifth Step - Validation

Check Resource States

Check ESM Server for readiness (takes 120 seconds or more):

```
kubectl rollout status statefulset mysm-smserver
```

Check ESM Agent for readiness (takes 120 seconds or more):

```
kubectl rollout status deployment mysm-smagent
```

Check Logs

Check ESM Server logs:

```
kubectl logs mysm-smserver-0 -f
```

Check ESM Agent logs:

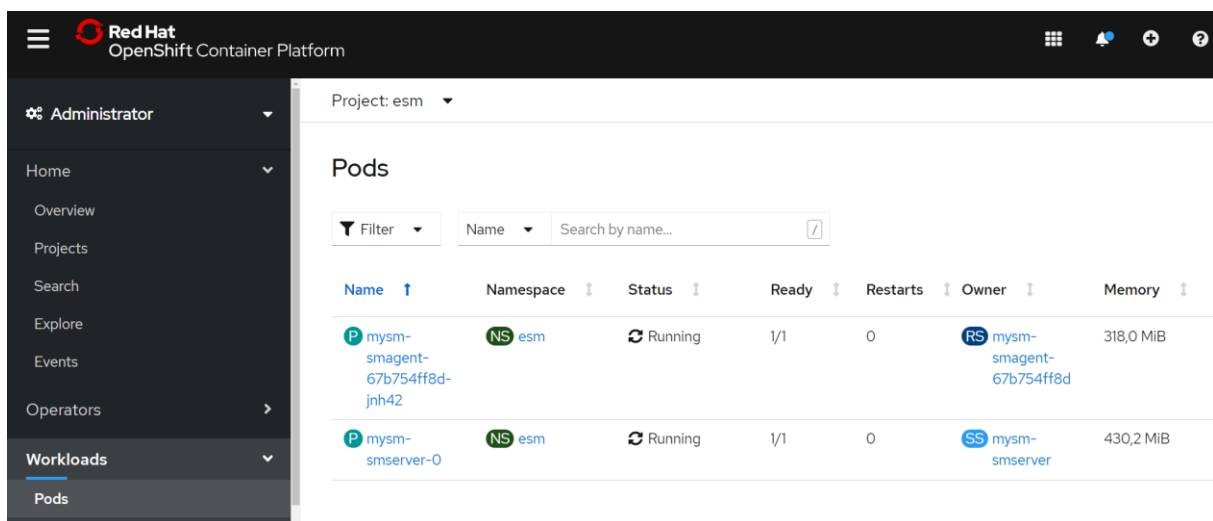
```
kubectl logs \
  $(kubectl get pod -n esm -l smagent-instance=mysm \
    -o=jsonpath='{.items[0].metadata.name}') -f
```

Show Pods

After a successful deployment the command `oc get pod -n esm` shows you the ESM Server and the ESM Agent Pods running and ready:

NAME	READY	STATUS	RESTARTS	AGE
mysm-smagent-67b754ff8d-jnh42	1/1	Running	0	5m22s
mysm-smserver-0	1/1	Running	0	5m22s

The running Pods can also be shown in the OpenShift Admin Console:



Project: esm

Pods

Filter Name Search by name...

Name ↑	Namespace ↑	Status ↑	Ready ↑	Restarts ↑	Owner ↑	Memory ↑
 mysm-smagent-67b754ff8d-jnh42	 esm	 Running	1/1	0	 mysm-smagent-67b754ff8d	318,0 MiB
 mysm-smserver-0	 esm	 Running	1/1	0	 mysm-smserver	430,2 MiB

Access ESM Server UI Using Port Forwarding

If your cluster is configured in a way that external access is not permitted or technically impossible, you can alternatively tunnel the access to the ESM Server UI through **kubect1** or **oc** to your local client machine:

```
oc port-forward -n esm mysm-smserver-0 8443:8443
```

As long as the shell of this running command is kept open, you will be able to access the UI from the machine you executed the command using URL: <https://localhost:8443>

If no browser is available on the local system, use curl command for a connection test (should return **200**):

```
curl -skL -o /dev/null https://localhost:8443 -w "%{http_code}\n"
```

Uninstallation

Uninstall the Helm release using:

```
helm uninstall mysm
```

Before running this command, make sure to have the correct namespace context configured (using **oc project**). A safer approach is to explicitly state the namespace in the uninstall command:

```
helm uninstall mysm -n esm
```

This will remove all resources managed by the Helm chart. This does not include:

- Manually created secrets using oc (like the access to the container registry or for the ESM admin password).
- The Persistent Volume of the ESM Server.

Although the creation of the ESM Server pods's Persistent Volume is initiated by the Helm chart, the chart does not manage the volume, as this is accomplished using Dynamic Volume Provisioning. The positive side-effect of this behavior is that after an accidental uninstallation, all persisted data (i.e. configuration done in the UI or collected monitoring data) is not automatically deleted, but still retained.

To completely erase all resources related to ESM from the cluster, run the following commands:

```
# Delete PVC (PV should be deleted automatically then)
RELEASE=mysm
oc delete pvc -n esm data-$RELEASE-smserver-0

# Delete admin password secret
oc delete secret -n esm my-sm-admin-pw

# Delete registry access secret
oc delete secret -n esm all-icr-io
```

Contact Information

If you have any questions, please contact us at ECM.SystemMonitor@cenit.com.

CENIT AG

Phone: +49 711 7825 30

Email: ECM.SystemMonitor@cenit.com